

Lesson 5: CSS Grid for Complex Designs

Mission Brief

A single-file row of ships is easy — but commanding an entire fleet formation, with rows AND columns aligned precisely, needs a real command grid. CSS Grid is the two-dimensional layout system built exactly for that. Today you'll lay out a full product/project gallery with precise rows and columns that reflow automatically as the screen changes size.

Learning Objectives

- Enable Grid with `display: grid` and define columns/rows
- Use `grid-template-columns` with `fr` units and `repeat()`
- Use `gap` in a grid context
- Build responsive grids with `auto-fit / auto-fill` and `minmax()`
- Explain when to choose Grid over Flexbox

What You'll Build

A responsive "Projects Gallery" grid of cards that automatically reflows from a multi-column layout on desktop to a single column on narrow screens — the direct basis for Project 02's project grid.

Prerequisites

Lesson 4: Flexbox concepts (container/item, gap), since Grid shares some vocabulary but is fundamentally two-dimensional.

Key Concepts

- Grid container and grid items
- `grid-template-columns` / `grid-template-rows`
- The `fr` unit
- `repeat()` and `minmax()`
- `auto-fit` vs `auto-fill`
- `gap` in Grid
- `grid-column` / `grid-row` spanning

Concept Explanation

Flexbox is one-dimensional — great for a single row or column. CSS Grid is **two-dimensional**: you define both columns and rows at once, and the browser places items into cells of that grid. You activate it with `display: grid` on a container, then describe the column structure with `grid-template-columns`.

The most powerful unit in Grid is `fr` (a "fraction" of the remaining space). `grid-template-columns: 1fr 1fr 1fr;` creates three equal-width columns that share available space evenly. You rarely want to hardcode a fixed number of columns for a responsive gallery, though — that's where `repeat()` and `minmax()` combine into one of the most useful lines in modern CSS:

```
grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
```

This reads as: "Fit as many columns as will comfortably hold at least 240px each, and let each column grow to fill any leftover space equally." As the browser window shrinks, columns automatically drop away and reflow — no media query required for basic reflow behavior (though you'll still combine Grid with media queries for bigger structural changes in Lesson 6).

`auto-fit` and `auto-fill` behave almost identically, but differ when there are fewer items than would fill a row: `auto-fill` keeps the empty column tracks (leaving gaps), while `auto-fit` collapses them so existing items stretch to fill the row. For most card galleries, `auto-fit` gives the more polished look.

`gap` (or the older `row-gap` / `column-gap`) adds spacing between grid cells, working exactly like Flexbox's `gap` — no more margin math.

Individual grid items can span multiple columns or rows using `grid-column: span 2;` or explicit line numbers like `grid-column: 1 / 3;` — useful for making a "featured" card twice as wide as its neighbors.

As a rule of thumb: reach for **Flexbox** when arranging items in a single row or column (a nav bar, a button group, a card's internal layout), and reach for **Grid** when you need a true two-dimensional layout (a photo gallery, a product catalog, an entire page's macro-layout of header/sidebar/content/footer). It's completely normal — expected, even — to use both together: Grid for the overall gallery, Flexbox for the layout inside each card.

Guided Coding

```

.projects-gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 24px;
  padding: 24px 0;
}

.project-card {
  background-color: #ffffff;
  border: 1px solid #dde3f0;
  border-radius: 10px;
  overflow: hidden;
  display: flex;          /* Flexbox handles the INSIDE of each card */
  flex-direction: column;
  transition: transform 0.2s ease;
}

.project-card:hover {
  transform: translateY(-4px);
}

.project-card__media {
  height: 160px;
  background: linear-gradient(135deg, #4361ee, #7209b7);
}

.project-card__body {
  padding: 16px;
  flex: 1;              /* body grows to fill remaining card height */
}

.project-card__body h3 {
  margin-bottom: 8px;
  color: #1a1a2e;
}

/* Make the first card a featured, wider highlight */
.project-card--featured {
  grid-column: span 2;
}

```

```

<div class="projects-gallery">
  <article class="project-card project-card--featured">
    <div class="project-card__media"></div>
    <div class="project-card__body">
      <h3>Orbit Tracker</h3>
      <p>A dashboard that visualizes live satellite positions using public open data.</p>
    </div>
  </article>
  <article class="project-card">
    <div class="project-card__media"></div>
    <div class="project-card__body">
      <h3>Star Chart</h3>
      <p>An interactive constellation map built with pure HTML and CSS.</p>
    </div>
  </article>
  <article class="project-card">
    <div class="project-card__media"></div>
    <div class="project-card__body">
      <h3>Mission Planner</h3>
      <p>A simple checklist app for planning multi-step class projects.</p>
    </div>
  </article>
</div>

```

Code Walkthrough

- `.projects-gallery { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); }` — creates as many 240px-minimum columns as fit, each sharing leftover space equally.
- `gap: 24px` spaces both rows and columns evenly.
- `.project-card { display: flex; flex-direction: column; }` — each card is *itself* a flex container, stacking its media and body vertically. This is Grid (macro layout) and Flexbox (micro layout) working together.
- `.project-card__media` uses a CSS `linear-gradient` as a placeholder visual instead of a broken/missing image — a real, valid, colorful background needing no external file.
- `.project-card:hover { transform: translateY(-4px); }` combined with `transition` gives a subtle lift animation on hover (previewed here, formalized in Lesson 7).
- `.project-card--featured { grid-column: span 2; }` makes that one card occupy two column tracks instead of one, creating visual hierarchy.

Student Mission

1. Build the `.projects-gallery` above with at least 5 cards describing fictional or real mini-projects.
2. Give each card a different `linear-gradient` combination for its `__media` block.
3. Resize your browser from wide to narrow and observe the column count changing automatically.

4. Make exactly one card `--featured` and confirm it visually spans two columns on wide screens.

Challenge

Add a nested grid inside one card's body (e.g. a 2-column mini "stats" grid showing "Language: HTML" / "Status: Complete") using `display: grid; grid-template-columns: 1fr 1fr;` scoped only to that inner container.

Experiment/Extension

Change `auto-fit` to `auto-fill` in `grid-template-columns` and reduce your gallery to only 2 cards.

Compare how the two remaining cards behave — with `auto-fit` they stretch to fill the row; with `auto-fill` empty invisible column tracks remain, so the cards may not stretch. Note the difference in a comment.

Common Mistakes

- Setting `display: grid` on the item instead of the parent container.
- Forgetting `minmax()` inside `repeat(auto-fit, ...)`, which can produce oddly tiny or zero-width columns.
- Using Grid for a simple single-row nav bar where Flexbox would be simpler.
- Confusing `grid-column: span 2` (relative span) with `grid-column: 2 / 4` (explicit line numbers) and mixing them incorrectly.
- Forgetting that `gap` in Grid applies between rows too, not just columns.

Debugging Tips

- Chrome/Firefox DevTools show a "grid" badge on grid containers in the Elements panel — click it to overlay grid lines, track numbers, and gaps directly on the page.
- The Firefox Grid Inspector (Layout panel) is especially good at showing named lines and track sizes.
- If columns aren't reflowing, confirm the container actually has `display: grid` applied by checking the Computed tab — a typo like `dispay: grid` fails silently.

Check Your Understanding

1. What is the core difference between Flexbox and Grid?
2. What does the `fr` unit represent?
3. What does `repeat(auto-fit, minmax(240px, 1fr))` do in plain English?
4. What's the difference between `auto-fit` and `auto-fill`?
5. How would you make one grid item span two columns?

Mini Quiz + Answer Key

Quiz 1. Which property defines a grid's columns? a) `flex-template` b) `grid-template-columns` c) `grid-columns` d) `column-count` 2. True or False: Grid is one-dimensional like Flexbox. 3. What unit represents a fraction of remaining space in Grid? 4. Which function lets you set a minimum and maximum size for a track? 5. What property makes an item occupy two grid columns?

Answer Key 1. b) `grid-template-columns` 2. False — Grid is two-dimensional. 3. `fr` 4. `minmax()` 5. `grid-column: span 2;`

Lesson Recap

- CSS Grid handles two-dimensional layouts (rows AND columns).
- `fr` units and `repeat(auto-fit, minmax(...))` create responsive grids with no media queries needed for basic reflow.
- `gap` spaces grid tracks evenly.
- Grid and Flexbox combine naturally: Grid for macro layout, Flexbox inside each item.
- `grid-column: span N` creates featured/wide items.

Homework

Build a responsive "Gallery" section on your Crew Profile page showing at least 6 fictional mission badges/achievements using the auto-fit grid pattern, each with a unique gradient background.

Portfolio Project Connection

This exact responsive card-grid pattern becomes the **projects grid in Project 02 (Responsive portfolio layout)** and the **product catalog grid in Project 03 (Product showcase site)**.